

Network CoPilot: Intent-Driven Network Configuration Updating for Service Guarantee

Rongxin Han, Jingyu Wang*, Haifeng Sun, Zengteng Jiang, Qi Qi*,
Zirui Zhuang, Yuan Zhang and Jianxin Liao

State Key Laboratory of Networking and Switching Technology,
Beijing University of Posts and Telecommunications, Beijing, China

Email:{rongxinhan, wangjingyu, hfsun, timmmm, qiqi8266, zhuangzirui, xiaolongtuan, liaojx}@bupt.edu.cn

Abstract—Updating network configurations not only requires satisfying new forwarding strategies, but also faces critical performance constraints brought on by various services. However, existing methods face significant limitations in interaction form, customization, and performance guarantees, making it challenging to keep pace with the rapid development of enterprise networks. Therefore, we propose a network copilot, a natural language intent-driven model for network configuration updating called *NLI2Conf*, which integrates graph and language models. We design network multimodal interfaces based on text attribute graphs to input network information in its raw format, including configuration files, topology, and performance metrics. This eliminates the need for additional specialized language design and learning. *NLI2Conf* understands forwarding strategies and performance constraints in natural language, and updates existing configurations, achieving alignment between intents and device behaviors. We provide a configuration collection process that mirrors the real network, and collect data samples from strategy intent to network configuration to facilitate *NLI2Conf*'s capability of configuration updating. Extensive experiments demonstrate that *NLI2Conf* can accurately interpret network information and strategy intent in raw format. The consistency between configuration and strategy intent reaches 96.6%, and the proposal reduces labor costs to $\frac{1}{26}$.

Index Terms—Intent-based network, Autonomous network, Network operation, Configuration updating, Language model

I. INTRODUCTION

Enterprise network operators exercise stringent control over the data flow, guiding network devices to perform correct forwarding behaviors to ensure transmission reliability and efficiency [1]. Network services represent the realization of the expectations of various stakeholders, such as users and network operators. These services are achieved through a combination of distributed protocols and network device behaviors [2]. Configuration acts as a bridge between devices and services, offering rich possibilities for current and future network services. However, the evolution of modern networks greatly increases complexity and scale, making the configuration process extremely tedious. It requires significant manual

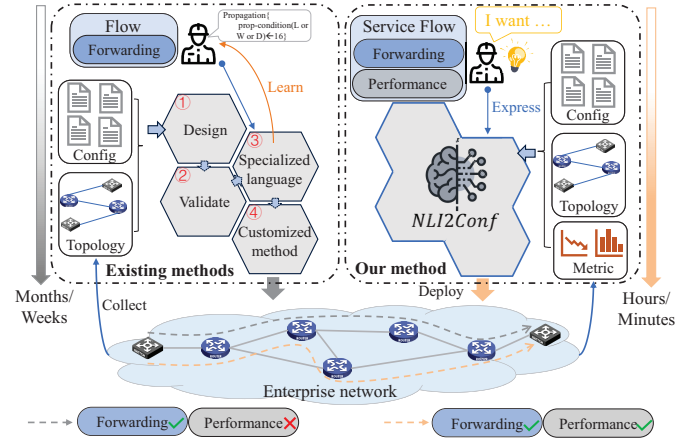


Fig. 1. Illustration of strategy intent-driven networks configuration updating. Comparison of existing methods with ours includes the intent expression form, scheme concept, and applicable scenario.

effort to learn and configure the network to meet all the different expected services [3].

Manually configuring devices to specify specific behaviors is no longer sustainable. Configuring networks with low thresholds and high efficiency to align high-level strategy intents with the forwarding behaviors of low-level devices becomes a significant challenge. This involves optimizing network behavior to remain consistent with the service consumption experience of users [2], [4].

Firstly, operators need to thoroughly understand networks as well as user requirements. However, learning various protocols, application program interfaces, complex configuration languages, and extensive documentation obstruct the efficiency of manual configuration [1]. As shown in Fig. 1, existing methods [3], [5]–[8] simplify intent expression and network description through custom domain-specific languages. These specialized languages rely on months-long manual design, are highly customized, and require extensive testing and validation. Every step involves meticulous information filtering and scheme design, making it often difficult to expand when encountering new problems. Operators need to invest weeks-long time to learn these specialized languages, which are difficult to use widely.

Secondly, for enterprise services, the configuration of devices must guarantee both the correctness and efficiency of for-

*Jingyu Wang and Qi Qi are the corresponding authors.

This work was supported in part by the National Natural Science Foundation of China under Grants (U23B2001, 62101064, 62201072, 62471055, 62321001, 62171057), the National Key R&D Program of China (2024YFE0200803, the Ministry of Education and China Mobile Joint Fund (MCM20200202, MCM20180101), the Fundamental Research Funds for the Central Universities (2024PTB-004).

warding behaviors, corresponding to the forwarding strategy of operators and the service quality of users. Existing works [5], [9]–[12] are limited to the former, lacking considerations of network performance, and neglecting its importance in service guarantee. These works cannot guarantee performance through configuration updating when services have stringent performance constraints.

Recently, natural language processing (NLP) technology has shown promising applications in programming languages, achieving significant success, particularly in code synthesis tasks [13], [14]. The input is the goal of the user, expressed in any formal language (typically described in natural language), and the output is a set of code that accomplishes the goal. This aligns well with the important question of how NLP can benefit network automation [15]. Based on this, a network copilot paradigm is proposed [2]: the input includes existing configurations, current network state, and strategy intents described by operators in natural language. The output is the updated configurations that are redeployed to align the intent with device behaviors. However, there are still some key challenges that need to be addressed in the implementation.

How to integrate multimodal raw format network information? The constraints and redundancy of specialized languages are detrimental to sustainable network configuration. Textual device configurations, graph-based network topologies, numerical performance metrics, and natural language intents are essential information for configuring networks, and they need to be effectively integrated as input. Existing large language models can only accept unstructured text descriptions, which do not fully meet the requirements of the multimodal input and result in the loss of unique and crucial structured information [16], [17]. Using a unified text window to input the information can lead to confusion and forgetfulness. Numerous device configurations in a large-scale network can exceed the token limit of the window [1].

How to achieve intent-configuration alignment driven by service guarantee? Configuration tasks based solely on qualitative forwarding strategies are insufficient to meet the performance constraints of critical services. Some work [5], [9], [18] transform network configuration into the logic constraint problem based on Boolean value discrimination. These approaches struggle to account for numerical metrics, thus lacking effective means to ensure performance through configuration updating. Some approaches [19] involve converting service guarantee requirements into forwarding strategies based on network performance, then synthesizing or updating network configurations with various methods [12], [20]. However, when the solving conditions in the preceding and subsequent stages are inconsistent or coupled, it results in unexpected network behavior, leading to unmet service requirements.

How to enhance understanding capability from forwarding to performance? The implementation of forwarding intents involves updating configurations of certain devices and then specifying their forwarding behaviors. This requires methods to understand or model the operational logic of various routing protocols as well as the mapping relationship between de-

vice configurations and forwarding behaviors [9], [12], [20]. However, such configuration inference problems are already NP-hard [21]. Furthermore, with the arrival of the service guarantee for performance intents, the model needs to understand the performance metrics, capturing the relationship between the metrics of links and overall service performance. The model needs to learn the inference capability from device configuration to forwarding behavior and then to service performance, as well as a comprehensive understanding of forwarding and performance. This undoubtedly raises higher requirements for the model.

To address the interaction barriers, high customization, and low strategy hierarchy faced by network configuration, we present a natural language intent-driven configuration updating model, named *NLI2Conf*. As a network copilot, it offers operators a low-threshold and flexible approach to updating configurations. The contributions of this paper are summarized as follows:

- Network multimodal interfaces are designed to directly integrate the device configuration, network topology, and performance metric with their raw format. Their semantic representations are obtained through self-attention mechanisms, thus avoiding the design and learning of additional specialized languages.
- A configuration updating model is proposed that can accept strategy intents in natural language as input. In addition to forwarding strategies, we innovatively consider performance constraints to update configurations for service guarantee, thereby aligning intents with device behaviors.
- A dataset collection process is provided that can collect batch data samples from strategy intents to configurations. A two-stage training method is introduced to enable the model to understand the semantics of configurations and metrics and learn configuration inference.

The remainder of this paper is organized as follows: Sec. II briefly introduces related work. Sec. III describes the background and establishes the theoretical model. Sec. IV details the workflow of the configuration updating model, the data collection, and the training method. Sec. V discusses and analyzes the results of the evaluation. Finally, Sec. VI concludes this paper.

II. RELATED WORK

A. Traditional Methods

Past research has made many efforts in configuration tasks aimed at large-scale networks, complex strategies, and network dynamics. Propane [6], Propane/AT [22], and Aura [3] design domain-specific languages to express operators' high-level strategies and propose various compilers that can generate low-level device configurations to improve operation efficiency. These methods avoid the high cost of manual configuration, but network engineers must be proficient in a new domain-specific language, which may not be widely used and may have flaws. Some works model the configuration synthesis as

a logical constraint problem, such as SyNET [5], Netcomplete [9], ConfigAssure [10], and AED [23] where high-level strategies are encoded as hard rules in the form of logical constraints. They rely on existing solvers [24] to synthesize precise network configurations, but due to the limited solving efficiency of existing solvers, configuring large-scale networks still requires a high time cost.

B. Graph Neural Network-based Methods

Inspired by the rapid development of deep learning and the hardware support for model acceleration, some configuration synthesis methods represented by graph neural networks (GNNs) have emerged. DeepBGP [11] is the first to use GNNs to represent network sketches and generate configuration parameters, but it is limited to border gateway protocol and cannot perform cross-protocol reasoning. To address this, work [12] based on the graph attention network proposes an extensible configuration synthesis model, providing a graph encoding approach for configuration sketch as input to reason configuration parameters, greatly improving the efficiency of large-scale network configuration synthesis. However, work [12] cannot synthesize completely satisfactory network configurations that can be deployed to production networks. NetRen [20] proposes a framework combining model inference and configuration repair, which can efficiently synthesize deployable configurations and for the first time provide optimization for network congestion based on configurations. However, with the emergence of various vertical services, providing differentiated service performance for users is equally important. The aforementioned works are limited to the consistency of configuration and forwarding strategy, neglecting the impact of network performance on the service guarantee.

C. Large Language Model-based Methods

NLP recently made astonishing progress in programming languages, quickly arousing researchers to explore its application in network configuration languages [15]. Lumi [7] is a natural language-based network interaction system that allows operators to express intents in natural language and convert them into domain-specific language Nile [8]. To break through the dependence on domain expertise and the limitations of manual design, Preconfig [25] uses pre-trained language models to automatically perform configuration tasks. The works [1], [26] test various configuration tasks with open large models (GPT-4, etc.), such as understanding and querying routing configurations, cross-vendor configuration translation, and operator requirement translation, which demonstrates the text understanding and translation capabilities of large language models. However, the single form of input and context limitations of language models become a disadvantage, making the processing of network topologies or verbose configurations difficult. The more valuable and challenging task lies in strategy intents-driven configuration tasks, which not only require the understanding capability but depend more on the inference ability for configurations.

<pre> hostname R10 interface Loopback0 ip address 1.1.1.11 255.255.255.255 negotiation auto interface GigabitEthernet1 ip address 1.0.25.2 255.255.255.0 negotiation auto ip ospf cost 19 router ospf 1 network 1.1.1.11 0.0.0.0 area 0 network 1.0.25.2 0.0.0.0 area 0 </pre>	<pre> route-map MODIFY permit 10 match ip address 1.1.1.18 set local-preference 2 set as-path prepend 1 set metric 1 router bgp 1 neighbor 1.0.41.1 remote-as 3 neighbor 1.1.1.5 remote-as 1 neighbor 1.1.1.5 update-source Loopback0 network 1.1.1.20 mask 255.255.255.255 neighbor 1.1.1.18 route-map MODIFY out </pre>
(a) OSPF	(b) BGP

Fig. 2. Example of OSPF/BGP configuration commands.

III. PROBLEM STATEMENT

A. Protocol and Configuration Background

The real-world network is a distributed system implemented by multiple routing protocols, determining the devices' forwarding behaviors through the configuration and route exchange of multiple peers [21]. We focus on two widely adopted routing protocols: Open Shortest Path First (**OSPF**) and Border Gateway Protocol (**BGP**).

OSPF is a link-state protocol used for routing within an autonomous system (AS) [27]. OSPF builds a network topology map by continuously exchanging link-state information between neighboring routers to calculate the best routing paths. BGP is a path vector protocol that operates between different ASes [28]. BGP routers exchange reachability information on the backbone network, announcing their presence to other networks and routers through BGP announcements. The received announcements are processed by filters, which may delete or modify these announcements. The best BGP announcement is selected based on local policies, further processed by export filters, and sent to other peers. Attributes of BGP announcements include local preference, AS path length, origin type, MED value, announcement type, OSPF cost to internal destinations, and others, to select the best path from multiple routing paths to the same prefix. The selection of the best BGP announcement typically follows the rules specified in the relevant standards [28].

We provide an example of Cisco configuration commands in Fig. 2. The configuration commands in Fig. 2(a) include hostname settings for network devices, interface configuration, IP address allocation, negotiation settings, OSPF cost settings, and routing protocol configuration. The configuration commands in Fig. 2(b) are used to determine forwarding paths based on BGP routing, including route match, modifying announcement attributes, and interacting with BGP neighbors.

Notably, we focus on the inferential relationship between configurations and forwarding planes, where the key parameters that truly impact the calculation of forwarding paths are of two types: 1) OSPF cost; and 2) BGP announcement attributes, as indicated by the blue markers in Fig. 2. Works [29], [30] verify that this setup applies to most real-world networks. Other basic configuration commands are relatively straightforward and can be implemented through methods such

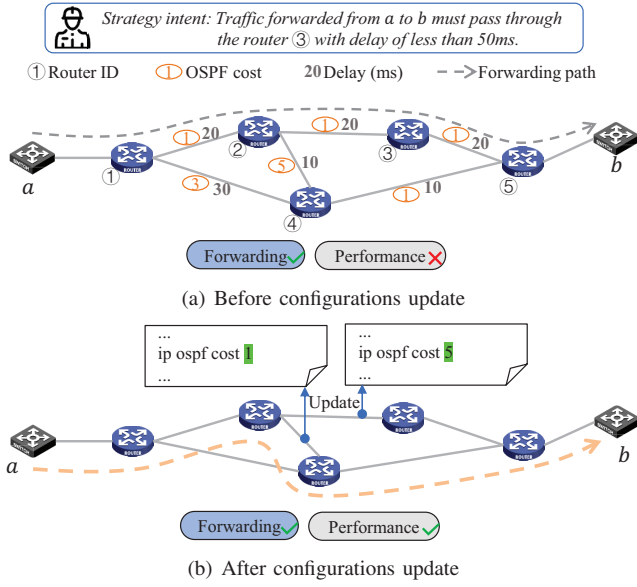


Fig. 3. A case study on configuration updating and the impact of forwarding strategy and performance constraint on configuration.

as template-based approaches [31] and generation models [1], [25], [26].

B. Theory and Problem Modeling

When configurations are deployed to each device, the primary concern is how the network forwards traffic, i.e., the forwarding behavior of the devices. The forwarding plane FWD is composed of the forwarding behaviors of the devices, determined through routing exchange and distributed computation based on the selected routing protocols $PROT$, and can be defined as [12]:

$$FWD = PROT(\varpi, \mathcal{G}). \quad (1)$$

For configuration parameters ϖ , the forwarding plane FWD obtained by applying routing protocols on the network topology $\mathcal{G}$ must meet the operator's forwarding specifications. Work [12] considers the forwarding specification $\mathcal{S} = \{\mathcal{R}_i\}$ as an input to the configuration problem with constraints, where $\mathcal{S}$ is a set of forwarding strategies, such as *forwarding*, *waypoint* and *isolation*, etc. Specifically, *forwarding* designates the next hop of devices, *waypoint* means that the path between two devices must pass through a specified device, and *isolation* refers to different source-destination flows that must not simultaneously pass through the specified device.

However, with the emergence of numerous critical applications, higher demands are placed on real-world networks. These networks must not only meet the enterprise's forwarding strategies to ensure communication security but also satisfy performance constraints to guarantee service quality. We provide an example in Fig. 3, illustrating the impact of configuration on forwarding strategies and performance constraints. Fig. 3(a) demonstrates a case where the forwarding path is calculated based solely on configuring routing devices with forwarding strategies. Configurations and the resulting

forwarding plane, synthesized using methods such as those in [9], [20], ensure that traffic from device a to device b must pass through router ③. Based on configurations in Fig. 3(a), the forwarding path $\{a \rightarrow ① \rightarrow ② \rightarrow ③ \rightarrow ⑤ \rightarrow b\}$ meets the forwarding strategy. However, when performance constraints are required, such as end-to-end delay, the specified forwarding path violates the performance constraint.

Fig. 3(b) provides an example of our research problem, assisting enterprise networks in meeting new strategies by updating the current configurations, particularly regarding performance constraints for critical services. The configurations marked in green in Fig. 3(b) are modified using our method, and the forwarding paths are recalculated based on routing protocols. It is not difficult to verify that the new forwarding paths meet both the forwarding strategy and performance constraint.

Therefore, when the configuration problem needs to comprehensively consider both forwarding strategies and performance constraints, the problem becomes more challenging. We extend the definition of problem (1) as follows:

$$FWD = PROT(\varpi, \mathcal{NET}). \quad (2)$$

For configuration parameters ϖ , the forwarding plane FWD obtained by applying routing protocols on the network $\mathcal{NET}$ with performance states must meet service-level requirements. We redefine the specification $\mathcal{S} = \{r_i^1, r_j^2\}$, where $\mathcal{S}$ is a set of forwarding strategies r_i^1 (*forwarding*, *waypoint*, *isolation*) and performance constraints r_j^2 (*delay*, *jitter*, *packet loss*). Further, $\mathcal{R}^1$ and $\mathcal{R}^2$ are modeled as binary functions, where $\mathcal{R}^1(r_i^1) = 1$ and $\mathcal{R}^2(r_j^2) = 1$ indicate that $\mathcal{NET}$ meets forwarding strategies and performance constraints, respectively; otherwise, they equal 0. The expanded configuration problem can be formulated as follows:

$$\begin{aligned} \varpi^* &= \arg \max_{\varpi \in \Phi(\varpi)} C(PROT(\varpi, \mathcal{NET}), \mathcal{S}) \\ &= \frac{(\sum_i \mathcal{R}^1(r_i^1) + \sum_j \mathcal{R}^2(r_j^2))}{|\mathcal{S}|}, \end{aligned} \quad (3)$$

where, $\Phi(\varpi)$ represents the search space of configuration parameters, and $C(PROT(\varpi, \mathcal{NET}), \mathcal{S})$ denotes the consistency of the network $\mathcal{NET}$ with respect to specification $\mathcal{S}$. Generally, for the configuration problem with an extremely low tolerance for errors, prior work [5], [9] achieve $C(PROT(\varpi, \mathcal{NET}), \mathcal{S}) = 1$, and ensure that the output configuration satisfies all specifications. However, these work face high time costs and poor timeliness, which is not conducive to a fast and scalable configuration workflow [5], [9].

We note that relaxing strict compliance with specifications, allowing configurations where $C(PROT(\varpi, \mathcal{NET}), \mathcal{S}) \leq 1$, while maximizing the value of C as a measure of configuration quality, is considered practical [12]. Learning-based models face challenges and require a trade-off between generalization and configuration quality. Subsequent configuration repairs provide a feasible path for this approach [20]. Therefore, the configuration updating model learns to infer approximately

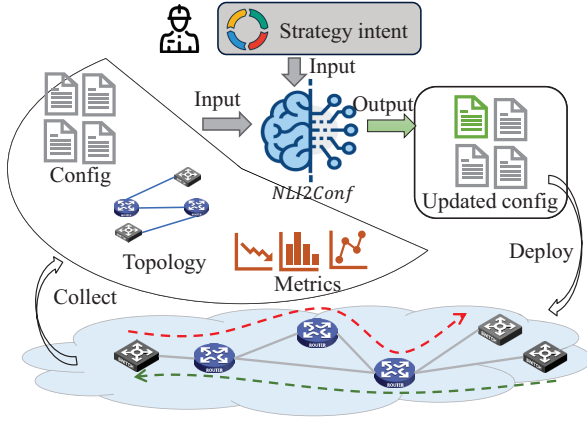


Fig. 4. The workflow, input and output of *NLI2Conf*.

feasible configurations, producing a configuration for a given network $\mathcal{NET}$ and specification $\mathcal{S}$.

$$\begin{aligned} \varpi^* &= \text{NLI2Conf}(\mathcal{NET}, \mathcal{S}), \\ \text{s.t. } C(\text{PROT}(\varpi, \mathcal{NET}), \mathcal{S}) &\text{ is high.} \end{aligned} \quad (4)$$

IV. NETWORK COPILOT: *NLI2Conf*

A. Workflow

As shown in Fig. 4, we develop a natural language intent-driven configuration updating model, *NLI2Conf*. This model updates the current configurations based on the existing configuration file, link state, network topology, and natural language intent of the operator, ensuring the satisfaction of forwarding strategies and performance constraints. It is worth noting that to ensure the generality of the model, we do not provide any additional design for these inputs. The model inference is implemented based on the raw form of them. We describe these inputs as follows.

They are divided into two parts, from the network side including device configuration file, link state, and network topology, and from the operator side instruction which is natural language intent.

Configuration file: As shown in Fig. 2, the textual device configuration is directly used as input of network nodes. Configuration files can usually be viewed and obtained through commands provided by vendors. We denote the textual configuration data of node i with the symbol $\mathbb{C}_i$. Specifically, this is a set containing K_i tokens, $\mathbb{C}_i = \{t_{i,1}, \dots, t_{i,K_i}\}$. The set of configuration files for all devices is $\mathbb{D} = \{\mathbb{C}_i, \forall i \in \mathcal{V}\}$, where $\mathcal{V}$ is the set of network nodes.

Link state: In addition to ensuring forwarding strategies, we also consider how to meet performance constraints based on the current network performance. Therefore, link states, which are directly related to routing optimization, are taken as input and modeled as edges and edge attributes in the graph. The edge between nodes i and j can be defined as:

$$\varepsilon_{i,j} = \begin{cases} (\eta_{i,j}^1, \eta_{i,j}^2, \eta_{i,j}^3), & \text{interface between } i \text{ and } j, \\ \vec{0}, & \text{else.} \end{cases} \quad (5)$$

Here, the three key performance metrics, $\eta_{i,j}^1$, $\eta_{i,j}^2$, and $\eta_{i,j}^3$, are delay, jitter, and packet loss, respectively. We collect these metrics through network monitoring.

Network topology: The input graph $\mathcal{G}$ can be represented as a tuple $\{\mathbb{D}, \mathcal{V}, \mathcal{E}\}$, which is a text attribute graph (TAG).

Natural language intent: Related studies and network operators themselves confirm the effectiveness of command interfaces, as they prefer to describe their network operation ideas using natural language [32]. This form is free and efficient, avoiding the additional learning costs associated with specialized languages. We provide natural language descriptions of operators' forwarding strategies and performance constraints, denoted as $\mathcal{I}$. Below, we provide an example sentence of natural language descriptions for the *waypoint* strategy and *delay* constraint:

"We need to forward the traffic from node R65 to node R12 through node R21. We need to ensure that the delay of the traffic from node R7 to node R34 is less than 10 ms."

Configuration updating: Our objective is to update existing routing configurations to meet the new strategies of the operator in the forwarding plane and performance. Consistent with the description in Sec. III-A, we focus on key configuration parameters of OSPF and BGP. Below, we provide an example output of *NLI2Conf* for the configuration updating.

"[{ 'type': 'ospf weight', 'location': (R3, R0), 'old value': { 'cost': 31 }, 'new value': { 'cost': 25 } }, { 'type': 'bgp route', 'location': R12, 'old value': { 'local preference': 88, 'med': 1 }, 'new value': { 'local preference': 30, 'med': 1 } }]"

This is a textual response regarding intents, essentially an updating abstract $\mathcal{A}$ that includes configuration content, updating location, original parameters, and target parameters, enabling precise configuration modifications. Therefore, based on the definition of configuration updating task using the language model: Given a TAG $\mathcal{G}$ and corresponding configuration updating for strategy intents, *NLI2Conf* involves effectively modeling structural data from configuration files $\mathbb{D}$, link states $\mathcal{E}$, network topology and intent $\mathcal{I}$ to inference configuration updating $\mathcal{A}$.

B. Configuration Updating Model

We illustrate the configuration updating model in Fig. 5, which is based on large language models and integrates graph models to enhance graph reasoning capabilities. We primarily address two issues: (1) how to input a TAG enriched with various types of raw network information and natural language intents, and (2) how to effectively integrate network information to update configurations with language models.

Firstly, we employ the *Tokenizer* algorithm [33], which is a fundamental component of natural language models. Based on the *Tokenizer*, text can be converted into independent token lists, which are then transformed into input vectors in a format understandable by the model.

$$\mathbb{T}_i^C = \text{Tokenizer}(\mathbb{C}_i), \forall i \in \mathcal{V}, \quad (6)$$

$$\mathbb{T}^I = \text{Tokenizer}(\mathcal{I}). \quad (7)$$

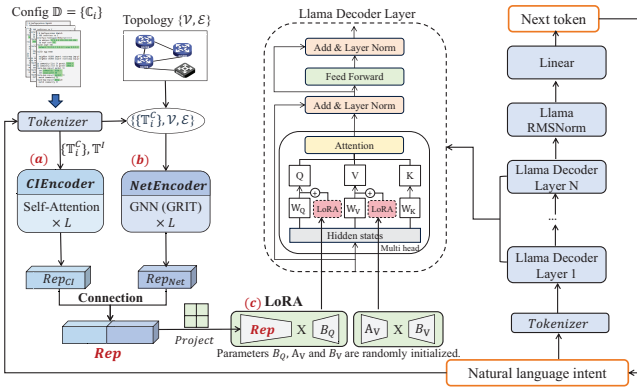


Fig. 5. The model architecture of *NLI2Conf*, and the key three modules: (a)*CIEncoder*, (b)*NetEncoder*, (c)*LoRA*.

As shown in Fig. 5(a), we propose an encoder called *CIEncoder* to capture the relationship between configurations and natural language intents. The *CIEncoder* is composed of L stacked layers of self-attention [34], so the context representation vector Rep_{CI} for configurations and intents is given by

$$Rep_{CI} = \sum_{i \in \mathcal{V}} CIEncoder(\mathbb{T}_i^C, \mathbb{T}^I) \in \mathbb{R}^{d_{seq} \times d_{hid}}, \quad (8)$$

where d_{seq} and d_{hid} are the length of token sequences and the dimension of hidden layers.

Fig. 5(b) shows the encoder for network embeddings, *NetEncoder*, responsible for aggregating three types of inputs: configuration files, link states, and network topology. *NetEncoder* consists of L layers stacked GNNs that rely on message-passing mechanisms. Generally, the message aggregation of node i in the l -th layer is achieved through exchanging messages with its neighbors $\mathcal{N}_i$ as follows:

$$m_i^{l+1} = \sum_{j \in \mathcal{N}_i} M_l(h_i^l, h_j^l, \varepsilon_{i,j}), \quad (9)$$

$$h_i^{l+1} = U_l(h_i^{l+1}, m_i^{l+1}), i \in \mathcal{V}, \quad (10)$$

where node features $\mathbb{T}_i^C$ are initialized as hidden states h_i^0 . M_l and U_l represent the message aggregation and update functions, respectively. We employ the state-of-the-art GRIT [35] as the core model of *NetEncoder*. Finally, we aggregate the representation of the entire network using an average function to compute Rep_{Net} .

$$H_i = (h_i^0 \parallel h_i^1 \parallel \dots \parallel h_i^{L-1}), \quad (11)$$

$$Rep_{Net} = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} H_i \in \mathbb{R}^{d_{seq} \times d_{hid}}, \quad (12)$$

where $\parallel$ denotes concatenation operation. (11) is to avoid the forgetting of node information caused by long-range aggregation. Furthermore, we integrate Rep_{CI} and Rep_{Net} as the multimodal representation vector of the network, with the process as follows:

$$Rep = (Rep_{CI} \parallel Rep_{Net}) \cdot W_{project}, \quad (13)$$

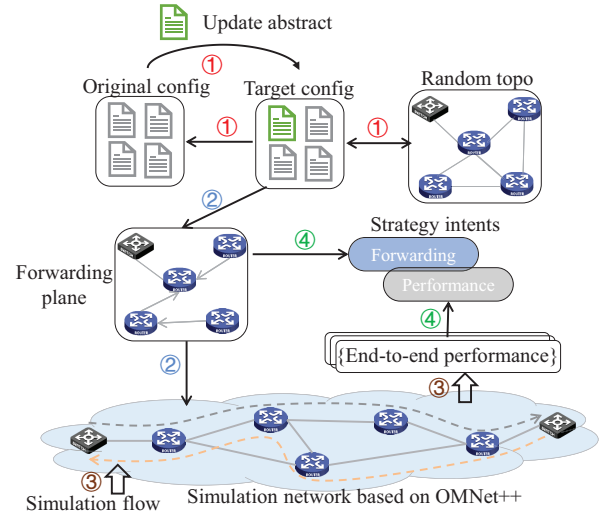


Fig. 6. The process of dataset collection.

$W_{project} \in \mathbb{R}^{2 \times d_{hid} \times d_{llm}}$ is a trainable parameter aligning with the embedding space of the language model, and d_{llm} is the hidden layer dimension of the large language model.

As shown in Fig. 5(c), to efficiently integrate the multimodal network representation Rep with the large language model, we adopt a fine-tuning approach called Low-Rank Adaptation (LoRA) [36]. The core idea is to load a small number of trainable parameters specific to the task into the query Q and value V layers of the attention mechanism, while freezing the pretrained shared parameters in the large language model. The construction process of the new mapping parameters W'_Q and W'_V for the Q and V layers is as follows:

$$W'_Q = W_Q + B_Q \cdot Rep, \quad (14)$$

$$W'_V = W_V + B_V \cdot A_V, \quad (15)$$

where W_Q and W_V are the pre-trained shared parameters in the large language model, and B_Q , B_V , and A_V are newly loaded trainable parameters specific to the configuration updating task. The LoRA fine-tuning approach for integrating Rep enables enhanced understanding of multimodal network information by the language model while maintaining its strong linguistic capabilities, thereby reducing training and task-switching overheads. We employ Llama 2, which has a parameter count of 7B, as the core language model [37].

C. Dataset Collection

Training a feasible configuration updating model requires a large amount of high-quality datasets. However, collecting high-quality network configuration datasets is a valuable yet challenging task. The reasons include: 1) publicly available data is certainly not abundant, and 2) the requirements for configuration data are much more stringent than those for natural language. The work [20] provides a foundational data generation idea for deep learning-based methods. Unlike [12], [20], we need to consider not only the forwarding behavior of the device that constitutes the forwarding plane but also how the link states affect the forwarding performance. To this end,

we propose a data collection process capable of generating data samples in bulk, as shown in Fig. 6.

Firstly, we build the network environment using a topology dataset. Based on the configuration described in Sec. III-A, we randomly generate configurations for each network device. During training, these configurations are used as the target configurations to guide the model toward the correct configuration updating. Then, they are modified randomly to obtain the original configurations. These changes are distilled into updating abstracts $\mathcal{A}$ described in Sec. IV-A. The original configuration serves as the input. Secondly, the forwarding plane is computed based on routing protocols and target configurations, and it is deployed to a simulation network based on OMNet++. This simulation network is a one-to-one mapping to the network topology in the first step.

Thirdly, we perform traffic simulation in the network based on OMNet++. Performance data of link states can be captured. Then we can calculate the end-to-end performance of forwarding paths between any two devices based on the forwarding plane and link states. Finally, we extract forwarding strategies from the forwarding plane [12]. From the end-to-end performance data, we extract performance constraints. For instance, if the end-to-end delay between two devices is 60 ms, the delay constraint is set to require that the end-to-end traffic is less than 100 ms. Then, the forwarding strategies and performance constraints are converted into the natural language intents form in Sec. IV-A.

Through the above process, we can collect complete data samples including {original configurations, target configurations, link states, network topology, strategy intents, updating abstract}. It is worth noting that we extract intents by reverse-engineering from target configurations. The purpose is to ensure that there is always a feasible configuration under the constraints of intents. However, the target configuration is not the only configuration that satisfies intents; it is just one of the feasible configurations. Furthermore, a large number of data samples form a dataset used for training.

D. Two-stage Training

Configuration updating is a highly challenging reasoning task, especially as we aim to maintain the model's generality to support input in its raw form. This undoubtedly places high demands on the model's capabilities, primarily divided into two parts: (1) semantic understanding of configuration and structure; and (2) aligning natural language intents with configurations and implementing accurate updating.

First, we adopt a commonly used masked pre-training technique in NLP [38], to enhance the semantic understanding capabilities of the language model through self-supervised learning. The core idea involves the model predicting the probability distribution of the masked token based on contextual tokens and comparing it with the ground truth. However, unlike typical contextual semantics self-supervised learning, routing configurations involve structurally related semantics that are challenging to infer solely from context [39]. Therefore, during training, the representations obtained by *CIEncoder* and

TABLE I
PARAMETERS OF SIMULATION NETWORKS BASED ON OMNet++.

Parameter	Value
Bandwidth(Kbps)	[20, 10 ⁶]
Loss rate(%)	[0.1, 40]
Delay(ms)	[1, 10 ³]
Flow size(byte)	4096
Number of flows	[4, 200]
Interval time(s)	Exponential(5)

NetEncoder continuously integrate with those modeled by the language model for configuration reasoning, naturally aligning these two aspects throughout the process.

We use textual configuration data from each device to train *NLI2Conf*. For the TAG $\mathcal{G}$, all tokens from configurations of device i , $\mathbb{C}_i = \{t_{i,1}, \dots, t_{i,K_i}\}$ are used as supervision signals during training. For the k -th token, *NLI2Conf* first extracts its masked tokens $\mathbb{C}_{i,k}^{mask} = \{t_{i,1}, \dots, t_{i,k-1}, mask, t_{i,k+1}, \dots, t_{i,K_i}\}$. Then, the encoder models the network representation Rep and combines it with the contextual tokens $\mathbb{C}_{i,k}^{mask}$ to predict the probability distribution of the masked token, where the ground truth value is token $t_{i,k}$. Therefore, the objective of pre-training is to minimize the cross-entropy loss L_{CE} between the predicted probability distribution and the ground truth distribution [38].

$$\mathcal{L}_{i,k}^1 = L_{CE}(NLI2Conf(\mathcal{G}, \mathbb{C}_{i,k}^{mask}), t_{i,k}), \quad (16)$$

$$\min_{\theta_{CI}, \theta_{Net}, \theta_{LoRA}} \sum_{i \in \mathcal{V}} \sum_{k \in \mathcal{K}_i} \mathcal{L}_{i,k}^1, \quad (17)$$

where, θ_{CI} , θ_{Net} , and θ_{LoRA} are the parameters of the *CIEncoder*, *NetEncoder*, and the training matrices loaded based on LoRA, respectively.

Secondly, based on the dataset collected in Sec. IV-C, *NLI2Conf* undergoes fine-tuning training to achieve configuration updating. For n -th data samples $\{\mathcal{G}_n, \mathcal{I}_n, \mathcal{A}_n\}$, we input a TAG $\mathcal{G}_n$, and the operator's intent $\mathcal{I}_n$ into the model. The model outputs the configuration updating abstract, while the correct updating abstract $\mathcal{A}_n$ serves as the label to guide the model. Therefore, we also leverage the cross-entropy loss function to train the model:

$$\mathcal{L}_n^2 = L_{CE}(NLI2Conf(\mathcal{G}_n, \mathcal{I}_n), \mathcal{A}_n), \quad (18)$$

$$\min_{\theta_{CI}, \theta_{Net}, \theta_{LoRA}} \sum_n \mathcal{L}_n^2. \quad (19)$$

It is worth noting that in (17) and (19), the parameters involved in training are θ_{CI} , θ_{Net} and θ_{LoRA} , while the parameters of the large language model Llama2 are frozen. Therefore, the proportion of trainable parameters is very small, approximately 0.98%, significantly reducing training costs and enhancing operational efficiency during the deployment of new configurations.

V. EVALUATION

A. Experiment Setup

Dataset. We use the NetworkX tool to randomly generate topology sets as the network environment. To evaluate the

TABLE II

EXPERIMENTAL RESULTS OF DIFFERENT MODELS AT VARIOUS NETWORK SCALES, INCLUDING TEXT ACCURACY, STRATEGY CONSISTENCY, AND FULL SATISFACTION RATE.

Metrics(%)	Scale	Models			
		GPT-4o	GraphLLM	GraphAdapter	Ours
TA	14	4.0	51.4	63.0	94.2
	35	0	33.0	54.6	96.8
	70	0	30.2	48.3	97.2
	100	0	31.2	42.4	96.6
SC	14	5.4	52.2	63.9	94.8
	35	0	33.8	54.9	97.3
	70	0	31.1	49.1	97.5
	100	0	32.5	44.2	96.6
FSR	14	0	51.4	63.0	94.2
	35	0	33.0	54.6	96.8
	70	0	30.2	48.3	97.2
	100	0	31.2	42.4	96.6

effectiveness of the model, we set up configuration tasks for different network scales. Topologies with 14, 35, 70, and 100 nodes are provided. From the dataset collection process described in Sec. IV-C, we extract two types of intents: (i) forwarding strategies, which include *forwarding*, *waypoint*, and *isolation*, and (ii) performance constraints, which include *delay*, *jitter*, and *packet loss*. We provide different quantities of policy intents to differentiate the complexity of tasks, with the number ranging from 3 to 9. The size of the collected dataset is 7×10^3 and it is divided into the training, validation, and testing set in a ratio of 10:1:3.

Experimental Platform and Model Settings. All our experiments are conducted on a machine equipped with a Xeon Silver 4210R processor, 128 GB of RAM, and an A100 GPU with 80 GB of VRAM. The number of layers L for the *CIEncoder* and *NetEncoder* is 4. The token sequence length d_{seq} is 512, and the hidden layer dimension d_{hid} is 768. The hidden layer dimension d_{ilm} in Llama2 is 4096. Other model parameters can be set according to the settings in [37], [39], [40]. We adopt the Adam optimizer to train *NLI2Conf*, with a batch size of 4, an epoch size of 2, a learning rate of 5×10^{-5} , and a total of 2.5×10^3 training steps. To test the configuration effectiveness, a simulation environment based on the open-source tool OMNet++ is provided for implementing data collection and configuration deployment. Specific parameter settings are shown in Tab. I.

Evaluation Metrics and Comparisons. We provide three metrics: text accuracy (TA), strategy consistency (SC), and full satisfaction rate (FSR). Based on (17) and (19), TA compares the textual similarity between the model-generated configuration updating abstracts and the labeled abstracts, evaluating the ability to understand the syntax and semantics of intents, configurations, and topologies. Based on (4), SC refers to the proportion at which the configurations satisfy strategy intents, assessing the capability to understand intents and update configurations correctly. FSR refers to the proportion of tasks where the consistency of the updated configurations reaches 100%. It is used to evaluate the reduction in labor cost and improvement in operation efficiency. We compare state-of-the-art graph and language models, including GraphLLM [40],

TABLE III

CONSISTENCY RESULTS FOR DIFFERENT STRATEGY INTENTS, INCLUDING FORWARDING STRATEGY AND PERFORMANCE CONSTRAINT.

Intent type(%)	Scale	Models		
		GraphLLM	GraphAdapter	Ours
Forwarding	14	53.4	64.5	94.9
	35	34.7	55.3	97.5
	70	31.4	49.5	97.5
	100	32.6	46.0	96.5
Performance	14	51.8	63.5	94.3
	35	33.4	54.2	97.3
	70	30.8	49.0	97.5
	100	32.3	43.6	96.6

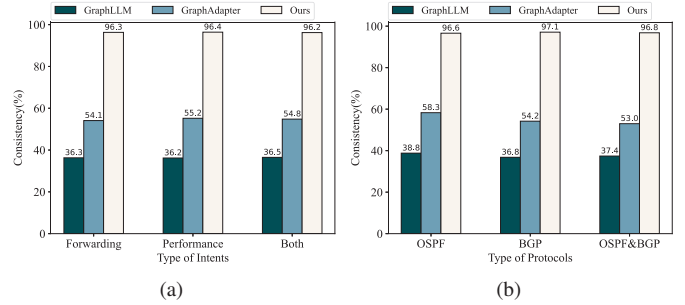


Fig. 7. Consistency results on tasks differentiated based on intents (a) and protocols (b).

GraphAdapter [39], and GPT-4o [41]. We train GraphLLM and GraphAdapter on the same dataset to ensure consistent comparison conditions. Since GPT-4o does not have a specific input module for the network, we use JSON-formatted text to input multimodal network information.

B. Experimental Results

Tab. II illustrates the results of text accuracy, strategy consistency, and full satisfaction rate across different network scales. As scale increases, configuration tasks become notably more complex, demanding greater graph reasoning capabilities of models. Our model demonstrates significant advantages in all metrics, with text accuracy, strategy consistency, and full satisfaction rate improving by 44.1%, 43.5%, and 44.1%, respectively. While other models experience performance degradation with increasing scale, our model maintains high-performance levels, achieving three metrics of 96.2%, 96.6%, and 96.2%, respectively. This indicates our model's strong scalability for large-scale networks. It is worth noting that the more configurations that fully satisfy all intents after modification, the less labor cost is required from operators. Only 3.8% of the configurations need to be reviewed and modified, which improves operational efficiency by $26\times$. Due to its lack of a dedicated structural input module, GPT-4o performs poorly in basic structural semantic understanding and advanced graph reasoning tasks, making it challenging to address configuration tasks effectively. And we no longer report the experimental results regarding GPT-4o.

Fig. 7(a) reports the results of distinguishing between types of intents, including considering forwarding strategies and performance constraints separately, as well as both together. Previous work is limited to forwarding strategies, and the

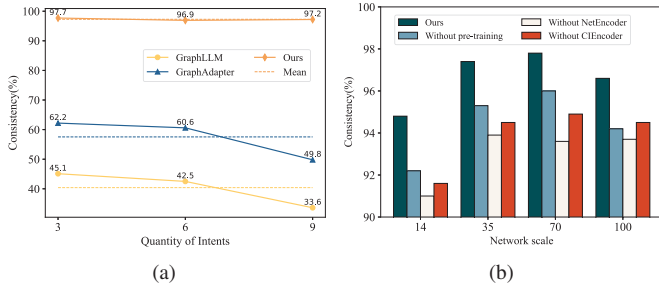


Fig. 8. (a) Consistency results for different numbers of strategy intents. (b) Ablation study on the key module and training phase.

TABLE IV
COMPARISONS OF TRAINING COST AND TRAINING SCALE.

Metrics	GraphLLM	GraphAdapter	Ours
VGA memory(GB)	72	69.75	52.75
Scale of training parameters(M)	93.3	82.5	66.7
Percentage of training parameters(%)	1.36	1.21	0.98

addition of performance constraints increases the complexity of the problem. In addition to forwarding strategies and routing protocols, analyzing performance metrics is necessary for the service guarantee. We find that consistency results are weaker when both forwarding strategies and performance constraints are considered compared to the others. Compared to other models, our model improves by 41.6%. Tab. III presents the consistency results of forwarding strategies and performance constraints across different network scales. It can be observed that ours maintains the advantage across different types of intents. By incorporating performance constraints based on configurations and network conditions, we enhance the competitiveness of our deployments in enterprise networks.

Fig. 7(b) presents the consistency results involving different protocols, including configurations with only OSPF or BGP, as well as configurations involving both. Configuring tasks about a single protocol are easier, with the consistency of models significantly higher compared to tasks involving multiple protocols. Our model maintains a consistency of 96.8% across all tasks, indicating superior protocol comprehension compared to the other models. This adaptability is particularly advantageous in real networks supporting multiple protocols.

Fig. 8(a) shows consistency results across different numbers of intents. We vary the number of intents from 3 to 9. As observed in (4), increasing the number of intents compresses the solution space of configurations, thereby increasing the difficulty for models to reason feasible configurations. Consequently, all models experience performance degradation as intent increases. Ours maintains consistency above 39.7%, demonstrating significant performance advantages. And it can accurately implement multiple intents simultaneously, and is suitable for operational scenarios with complex strategies.

Fig. 8(b) illustrates ablation experiments validating the effectiveness of the input module and pre-training process. We conduct comparative experiments by replacing the *NetEncoder* and *CIEncoder* with multilayer perceptrons of the same scale,

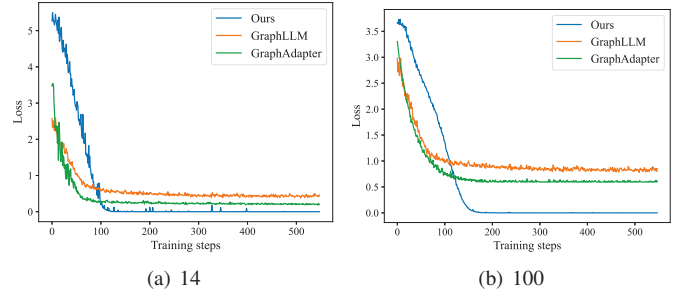


Fig. 9. Loss variation curve for training with network scales of 14 and 100.

resulting in a decrease of 3.6% and 2.8% in consistency. This underscores the importance of the *NetEncoder* and *CIEncoder* in the configuration updating task. Additionally, when omitting pre-training as described in Sec. IV-D and comparing, we find a decrease of 2.2%. This highlights that semantic understanding of structure and configuration forms the foundation of network configuration reasoning, thereby validating the effectiveness of our proposed module and training method.

Tab. IV reports on the cost and scale during training. VRAM consumption is significantly lower compared to other models. Our model compresses training costs by 24.4%, offering network operators a low-cost network configuration updating model. Fig. 9 reports the training loss curves for network scales of 14 and 100. Our model achieves better training results, and enables a better understanding of the mapping between configurations and forwarding behavior and performance. From Tab. IV, our model involves fewer training parameters, which reflects the superiority in terms of model design and learning capability for configuration updating.

VI. CONCLUSION

To address issues of interaction barriers, method customization, and low-level strategy expression, we propose a network copilot, which is a natural language intent-driven model for configuration updating, *NLI2Conf*. We expand the definition dimensions of the routing configuration problem to more comprehensively consider forwarding strategies and performance constraints for service guarantee. Network multimodal interfaces are designed to embed the configuration file, network topology, and link state in the form of text attribute graphs, allowing the acceptance of raw network information without the need for specialized language. We propose a configuration updating model based on graph and language models, incorporating the *CIEncoder* and *NetEncoder* to enable the semantic and syntax understanding of natural language intents and network information. A data collection process is provided and can gather data samples from various strategy intents to configurations. Based on the masked pre-training and LoRA fine-tuning approach, we efficiently implement configuration updating based on the open-source large language model. Our model can efficiently align intents with device behaviors through configuration updating, making it suitable for performance-demanding enterprise networks, and significantly improving operation efficiency.

REFERENCES

- [1] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostic, and M. Chiesa, "Netconfeval: Can llms facilitate network configuration?" *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, pp. 1–25, Jul 2024.
- [2] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Rfc 9315: Intent-based networking - concepts and definitions," USA, 2022.
- [3] S. Ramanathan, Y. Zhang, M. Gawish, Y. Mundada, Z. Wang, S. Yun, E. Lippert, W. Taha, M. Yu, and J. Mirkovic, "Practical intent-driven routing configuration synthesis," in *USENIX Symposium on Networked Systems Design and Implementation, NSDI*, Boston, MA, Apr 2023.
- [4] A. Leivadeas and M. Falkner, "A survey on intent-based networking," *IEEE Commun. Surv. Tutorials*, vol. 25, no. 1, pp. 625–655, Jan 2023.
- [5] A. El-Hassany, P. Tsankov, L. Vanbever, and M. T. Vechev, "Network-wide configuration synthesis," in *Computer Aided Verification - International Conference, CAV, Heidelberg, Germany, Jul 2017*.
- [6] R. Beckett, R. Mahajan, T. D. Millstein, J. Padhye, and D. Walker, "Don't mind the gap: Bridging network-wide objectives and device-level configurations," in *Proceedings of the ACM SIGCOMM*, Florianopolis, Brazil, Aug 2016.
- [7] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, W. Willinger, and S. G. Rao, "Hey, lumi! using natural language for intent-based network management," in *USENIX Annual Technical Conference, USENIX ATC*, Virtual Event, Jul 2021.
- [8] A. S. Jacobs, R. J. Pfitscher, R. A. Ferreira, and L. Z. Granville, "Refining network intents for self-driving networks," in *Proceedings of the Afternoon Workshop on Self-Driving Networks, SelfDN@SIGCOMM*, Budapest, Hungary, Aug 2018.
- [9] A. El-Hassany, P. Tsankov, L. Vanbever, and M. T. Vechev, "Netcomplete: Practical network-wide configuration synthesis with autocompletion," in *USENIX Symposium on Networked Systems Design and Implementation, NSDI*, Renton, WA, USA, Apr 2018.
- [10] S. Narain, G. Levin, S. Malik, and V. Kaul, "Declarative infrastructure configuration synthesis and debugging," *J. Netw. Syst. Manag.*, vol. 16, no. 3, pp. 235–258, Oct 2008.
- [11] M. Bahnasy, F. Li, S. Xiao, and X. Cheng, "DeepBGP: A machine learning approach for BGP configuration synthesis," in *Proceedings of the Workshop on Network Meets AI & ML*, B. Arzani and X. Jin, Eds., Virtual Event, USA, Aug 2020.
- [12] L. Beurer-Kellner, M. T. Vechev, L. Vanbever, and P. Velickovic, "Learning to configure computer networks with neural algorithmic reasoning," in *Annual Conference on Neural Information Processing Systems NeurIPS*, Louisiana, USA, Nov 2022.
- [13] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, pp. 85–111, Apr 2023.
- [14] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. Canton-Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," *CoRR*, vol. abs/2308.12950, Aug 2023.
- [15] Z. Ben-Houidi and D. Rossi, "Neural language models for network configuration: Opportunities and reality check," *Comput. Commun.*, vol. 193, no. C, pp. 118–125, Sep 2022.
- [16] S. K. Mani, Y. Zhou, K. Hsieh, S. Segarra, T. Eberl, E. Azulai, I. Frizler, R. Chandra, and S. Kandula, "Enhancing network management using code generated by large language models," in *ACM Workshop on Hot Topics in Networks, HotNets*, Cambridge, MA, USA, Nov 2023.
- [17] P. Sharma and V. Yegneswaran, "PROSPER: extracting protocol specifications using large language models," in *ACM Workshop on Hot Topics in Networks, HotNets*, Cambridge, MA, USA, Nov 2023.
- [18] B. Tian, X. Zhang, E. Zhai, H. H. Liu, Q. Ye, C. Wang, X. Wu, Z. Ji, Y. Sang, M. Zhang, D. Yu, C. Tian, H. Zheng, and B. Y. Zhao, "Safely and automatically updating in-network ACL configurations with intent language," in *Proceedings of the ACM SIGCOMM*, Beijing, China, Aug 2019.
- [19] C. Liu, M. Xu, Y. Yang, and N. Geng, "DRL-OR: deep reinforcement learning-based online routing for multi-type service requirements," in *IEEE Conference on Computer Communications, INFOCOM*, Vancouver, BC, Canada, May 2021.
- [20] R. Han, J. Wang, Q. Qi, H. Sun, C. Xu, Z. Wan, Z. Zhuang, Y. Yu, and J. Liao, "Netren: Service migration-driven network renaissance with synthesizing updated configuration," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS*, La Jolla, CA, USA, Apr-May 2024.
- [21] T. Schneider, R. Schmid, and L. Vanbever, "On the complexity of network-wide configuration synthesis," in *IEEE International Conference on Network Protocols, ICNP*, Lexington, KY, USA, Oct-Nov 2022.
- [22] R. Beckett, R. Mahajan, T. D. Millstein, J. Padhye, and D. Walker, "Network configuration synthesis with abstract topologies," in *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI*, Barcelona, Spain, Jun 2017.
- [23] A. Abhashkumar, A. Gember-Jacobson, and A. Akella, "AED: incrementally synthesizing policy-compliant and manageable configurations," in *International Conference on emerging Networking EXperiments and Technologies, CoNEXT*, Barcelona, Spain, Dec 2020.
- [24] S. Cai, B. Li, and X. Zhang, "Local search for SMT on linear integer arithmetic," in *Computer Aided Verification - International Conference, CAV, Haifa, Israel, Aug 2022*.
- [25] F. Li, H. Lang, J. Zhang, J. Shen, and X. Wang, "Preconfig: A pretrained model for automating network configuration," *CoRR*, vol. abs/2403.09369, pp. 1–14, Mar 2024.
- [26] R. Mondal, A. Tang, R. Beckett, T. D. Millstein, and G. Varghese, "What do llms need to synthesize correct router configurations?" in *ACM Workshop on Hot Topics in Networks, HotNets*, Cambridge, MA, USA, Nov 2023.
- [27] J. Moy, "OSPF version 2," *RFC*, vol. 2328, pp. 1–244, 1998.
- [28] Y. Rekhter, T. Li, and S. Hares, "A border gateway protocol 4 (BGP-4)," *RFC*, vol. 4271, pp. 1–104, 2006.
- [29] J. Luttringer, Q. Bramas, C. Pelsser, and P. Mérindol, "A fast-convergence routing of the hot-potato," in *IEEE Conference on Computer Communications, INFOCOM*, Vancouver, BC, Canada, May 2021.
- [30] P. François, M. Shand, and O. Bonaventure, "Disruption free topology reconfiguration in OSPF networks," in *IEEE International Conference on Computer Communications, INFOCOM*, Anchorage, Alaska, May 2007.
- [31] Z. Guo, F. Li, J. Shen, T. Xie, S. Jiang, and X. Wang, "Configreco: Network configuration recommendation with graph neural networks," *IEEE Netw.*, vol. 38, no. 1, pp. 7–14, Jan 2024.
- [32] R. Birkner, D. Drachler-Cohen, L. Vanbever, and M. T. Vechev, "Net2text: Query-guided summarization of network forwarding behaviors," in *USENIX Symposium on Networked Systems Design and Implementation, NSDI*, Renton, WA, USA, Apr 2018.
- [33] T. Kudo and J. Richardson, "Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Conference on Empirical Methods in Natural Language Processing, EMNLP*, Brussels, Belgium, Oct-Nov 2018.
- [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Annual Conference on Neural Information Processing Systems NeurIPS*, Long Beach, CA, Dec 2017.
- [35] L. Ma, C. Lin, D. Lim, A. Romero-Soriano, P. K. Dokania, M. Coates, P. H. S. Torr, and S. Lim, "Graph inductive biases in transformers without message passing," in *International Conference on Machine Learning, ICML*, Honolulu, Hawaii, Jul 2023.
- [36] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations, ICLR*, Virtual Event, Apr 2022.
- [37] T. Hugo, M. Louis, S. Kevin, A. Peter, A. Amjad, B. Yasmine, B. Nikolay, B. Soumya, B. Prajjwal, B. Shruti et al., "Llama 2: Open foundation and fine-tuned chat models," *CoRR*, vol. abs/2307.09288, Jul 2023.
- [38] Y. Chai, Y. Liang, and N. Duan, "Cross-lingual ability of multilingual masked language models: A study of language structure," in *Annual Meeting of the Association for Computational Linguistics, ACL*, Dublin, Ireland, May 2022.
- [39] X. Huang, K. Han, Y. Yang, D. Bao, Q. Tao, Z. Chai, and Q. Zhu, "Can GNN be good adapter for llms?" in *ACM on Web Conference, WWW*, Singapore, May 2024.
- [40] Z. Chai, T. Zhang, L. Wu, K. Han, X. Hu, X. Huang, and Y. Yang, "Graphllm: Boosting graph reasoning ability of large language model," *CoRR*, vol. abs/2310.05845, Oct 2023.
- [41] OpenAI, "GPT-4 technical report," *CoRR*, vol. abs/2303.08774, Aug 2023.